APUNTES. WINDOWS FORMS CON .NET

Objetivo:

Entender qué es WinForms, qué necesitas para programar con él y cómo construir una aplicación de escritorio paso a paso, aplicando buenas prácticas (eventos, validación, manejo de errores y arquitectura en capas).

1. ¿Qué es WinForms y para qué sirve?

WinForms (Windows Forms) es una tecnología de .NET para crear aplicaciones de escritorio para Windows con interfaz gráfica (botones, cuadros de texto, tablas, menús, etc.). Es muy útil para:

• Aprender eventos y lógica de interfaz (GUI).
• Crear herramientas internas, utilidades, aplicaciones sencillas de gestión.
• Prototipar interfaces rápidamente.

Características clave:

• Modelo basado en eventos (clic, cambio de texto, selección, etc.).

• Diseñador visual en Visual Studio (arrastrar y soltar controles).

• Controles clásicos de Windows (DataGridView, Combobox , TextEdit, etc.).

• Ideal para .NET Framework para Windows.

2. ¿Qué se necesita para programar WinForms?

Software recomendado (entorno típico de clase):

• Windows 10/11.

• Visual Studio Community.

• .NET Framework.

• (Opcional) SQL Server + SSMS si vas a hacer una app con base de datos.

Conocimientos previos:

• Variables, condicionales, bucles.

• Métodos y clases básicas.

• Conceptos básicos de POO (clases y objetos).

3. Crear tu primera aplicación WinForms (paso a paso)

3.1 Crear el proyecto

• Abre Visual Studio → Create a new project.

• Busca: “Windows Forms App” (elige .NET Framework).

• Asigna un nombre (ej.: AgendaWinForms) y una ubicación.

• Pulsa Create. Se generará un formulario inicial: Form1.

3.2 Entender qué ves en Visual Studio (lo importante)

• Diseñador (Form1.cs [Design]): donde colocas controles.

• Código (Form1.cs): donde programamos eventos y lógica.

• Solution Explorer: estructura del proyecto.

• Properties: configura propiedades del formulario o controles (Name, Text, Size…).

3.3 Primeros controles (ejemplo: Mini-calculadora)

En el Diseñador, arrastra y coloca:

• 2 TextBox (txtA, txtB)

• 1 Button (btnSumar) con Text=“Sumar”

• 1 Label (lblResultado) con Text=“Resultado: ”

Doble clic en el botón para crear el evento Click y añade este código:

```
private void btnSumar_Click(object sender, EventArgs e)
{
    if(!int.TryParse(txtA.Text, out int a) || !int.TryParse(txtB.Text, out
int b))
    {
        MessageBox.Show("Introduce dos números válidos.", "Validación",
```

```
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    int suma = a + b;
    lblResultado.Text = $"Resultado: {suma}";
}
```

Puntos didácticos:

• WinForms funciona por eventos: el código se ejecuta cuando ocurre algo (clic).

• Valida SIEMPRE lo que el usuario escribe (TryParse).

• Usa MessageBox para comunicar errores o avisos.

4. Controles más usados y cuándo usarlos

Label: Mostrar texto fijo o dinámico (estado, títulos, mensajes).

TextBox: Entrada de texto (nombre, email, búsqueda).

Button: Acciones: guardar, buscar, calcular, confirmar.

ComboBox: Selección de una opción (listas cortas/medias).

ListBox: Lista de elementos; útil para seleccionar varios.

CheckBox / RadioButton: Opciones booleanas o selección exclusiva.

DataGridView: Tabla para mostrar y editar datos (muy usado en CRUD).

DateTimePicker: Fechas sin errores de formato.

MenuStrip / ToolStrip: Menús y barras de herramientas.

Panel / GroupBox: Organizar interfaz y agrupar controles.

5. Propiedades importantes (Form y controles)

Name: Nombre del control en el código (ej.: btnGuardar).

Text: Texto visible (título del botón, etiqueta, formulario).

Enabled: Si el usuario puede interactuar (true/false).

Visible: Mostrar u ocultar el control.

Anchor / Dock: Cómo se adapta el control al cambiar el tamaño de la ventana.

TabIndex: Orden al moverse con TAB (muy importante en formularios).

AcceptButton: Botón que se pulsa al presionar ENTER.

CancelButton: Botón que se activa al presionar ESC.

6. Eventos (el corazón de WinForms)

Un evento es una “señal” de que algo pasó. En WinForms, programamos respuestas a esos eventos.

Eventos típicos:

• Button.Click → cuando el usuario hace clic.

• TextBox.TextChanged → cuando cambia el texto.

• Form.Load → al cargar el formulario (ideal para inicializar).

• DataGridView.CellClick / SelectionChanged → selección en la tabla.

• FormClosing → antes de cerrar (confirmaciones).

Ejemplo: confirmar al cerrar

```
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    var r = MessageBox.Show("¿Seguro que quieres salir?", "Salir",
        MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (r == DialogResult.No)
        e.Cancel = true;
}
```

7. Diseño de interfaz: buenas prácticas rápidas

• Nombra los controles con prefijos: btnGuardar, txtNombre, dgvClientes, lblError…

• Usa GroupBox/Panel para ordenar secciones.

• Configura Anchor/Dock para que la ventana sea “responsive”.

• Mantén formularios simples: un formulario = una responsabilidad principal.

• Evita lógica compleja en el formulario: muévela a las clases (capa de negocio).

8. Manejo de errores y validación

(Se revisará más adelante. Ahora vamos a lo sencillo primero. “Que funcione…..”)

Técnicas prácticas:

• TryParse para números y fechas.

• ErrorProvider para mostrar errores junto a cada campo.

• try/catch alrededor de operaciones que fallan (ficheros, BD, red).

Ejemplo con ErrorProvider (validación de un campo):

```
private readonly ErrorProvider _error = new ErrorProvider();

private bool ValidarNombre()
{
    if (string.IsNullOrWhiteSpace(txtNombre.Text))
    {
        _error.SetError(txtNombre, "El nombre es obligatorio.");
        return false;
    }
```

```
    _error.SetError(txtNombre, "");
    return true;
}
```

9. ¿Qué capas tiene una aplicación WinForms? (Arquitectura en capas)

Aunque WinForms es “solo” la interfaz, una aplicación bien organizada se estructura en capas para que el código sea mantenible.

9.1 Capas típicas (3 capas / 3-tier)

Capa de Presentación (UI): Formularios WinForms, controles, eventos. Solo coordina y muestra.

Capa de Negocio (Services): Reglas del dominio: validaciones, cálculos, casos de uso (GuardarCliente, CalcularPrecio…).

Capa de Datos (Repositories): Acceso a datos: BD, ficheros, API. Devuelve objetos/DTOs.

9.2 ¿Por qué usar capas?

• Separación de responsabilidades: cada parte hace lo suyo.

• Facilita pruebas (sobre todo en la capa de negocio).

• Reutilización: la misma lógica puede usarse luego en ASP.NET MVC.

• Mantenimiento: cambios en BD no deberían romper la interfaz (y viceversa).

9.3 Estructura de solución recomendada (Visual Studio)

• MiApp.WinForms (UI)

• MiApp.Domain (Entidades: Cliente, Producto, Pedido…)

• MiApp.Application (Servicios / Casos de uso)

• MiApp.Infrastructure (Datos: EF/ADO.NET, repositorios, ficheros)

Si quieres simplificar en clase: UI + Services + Data en 3 proyectos es suficiente.

10. Mini-proyecto guiado: sencillo (sin BD, con lista en memoria)

Objetivo: practicar WinForms + capas sin complicarnos con SQL.

Pantalla: alta y listado de productos.

Controles: txtNombre, txtPrecio, txtStock, btnAgregar, btnEliminar, dgvProductos.

10.1 Capa Domain (Producto)

```
public class Producto
{
    public string Nombre { get; }
    public decimal Precio { get; private set; }
    public int Stock { get; private set; }

    public Producto(string nombre, decimal precio, int stock)
    {
        if (string.IsNullOrWhiteSpace(nombre))
            throw new ArgumentException("Nombre obligatorio.");
        if (precio < 0) throw new ArgumentOutOfRangeException(nameof(precio));
        if (stock < 0) throw new ArgumentOutOfRangeException(nameof(stock));

        Nombre = nombre;
        Precio = precio;
        Stock = stock;
    }

    public void Reponer(int unidades)
    {
        if (unidades <= 0) throw new ArgumentOutOfRangeException(nameof(unidades));
        Stock += unidades;
    }

    public void Vender(int unidades)
    {
        if (unidades <= 0) throw new ArgumentOutOfRangeException(nameof(unidades));
        if (unidades > Stock) throw new InvalidOperationException("Stock insuficiente.");
        Stock -= unidades;
    }
}
```

10.2 Capa Application (Servicio)

```
public class ProductosService
{
    private readonly List<Producto> _productos = new();

    public IReadOnlyList<Producto> Listar() => _productos;

    public void Agregar(string nombre, decimal precio, int stock)
    {
        var p = new Producto(nombre, precio, stock);
        _productos.Add(p);
    }

    public void Eliminar(Producto producto)
    {
        if (producto is null) throw new ArgumentNullException(nameof(producto));
        _productos.Remove(producto);
    }
}
```

10.3 UI (WinForms): enlazar DataGridView y botones

```
private readonly ProductosService _service = new ProductosService();
private readonly BindingSource _bs = new BindingSource();

private void Form1_Load(object sender, EventArgs e)
{
    _bs.DataSource = _service.Listar().ToList(); // copia para refrescar fácil
    dgvProductos.DataSource = _bs;
}

private void RefrescarGrid()
{
    _bs.DataSource = _service.Listar().ToList();
}

private void btnAgregar_Click(object sender, EventArgs e)
{
    try
    {
        if (!decimal.TryParse(txtPrecio.Text, out var precio) ||
            !int.TryParse(txtStock.Text, out var stock))
        {
            MessageBox.Show("Precio o stock inválidos.");
            return;
        }

        _service.Agregar(txtNombre.Text, precio, stock);
        RefrescarGrid();
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void btnEliminar_Click(object sender, EventArgs e)
{
    if (dgvProductos.CurrentRow?.DataBoundItem is Producto p)
    {
        _service.Eliminar(p);
        RefrescarGrid();
    }
}
```

Notas del mini-proyecto:

• BindingSource ayuda a enlazar datos al DataGridView.

• La UI llama al servicio, y el servicio usa las entidades del dominio.

• La validación fuerte vive en el dominio (constructor/métodos).

11. Conexión a Base de Datos (idea general)

Cuando pasas de lista en memoria a BD, la capa de datos (DAL) se encarga de leer y guardar. Puedes usar ADO.NET (SqlConnection/SqlCommand) o Entity Framework (según el temario).

Recomendación didáctica: primero CRUD en memoria → luego CRUD con BD.

Si usas ADO.NET, separa:

• Repositorio/DAO: métodos como InsertarProducto, ObtenerProductos, EliminarProducto.

• DTOs o entidades: objetos que viajan entre capas.

12. Checklist: "lo que hay que saber" para dominar WinForms en clase

☐ Crear proyectos WinForms y navegar por Diseñador / Código.

☐ Usar controles básicos (Label, TextBox, Button, ComboBox).

☐ Programar eventos (Click, Load, TextChanged, FormClosing).

☐ Validar entradas (TryParse, ErrorProvider). Manejar errores con try/catch y mensajes adecuados.

☐ Organizar el código en capas (UI / Negocio / Datos).

☐ Usar DataGridView para listar y seleccionar datos.

☐ Buenas prácticas: nombres, separación de lógica, refresco de interfaz.

13. Ejercicios propuestos (para evaluar)

• Ejercicio 1 Calculadora con +, -, *, / (validación + mensajes).

• Ejercicio 2 Formulario de registro (Nombre, Email, Edad) con ErrorProvider.

• Ejercicio 3 CRUD en memoria con DataGridView (Producto o Cliente).

• Ejercicio 4 CRUD con BD: alta/baja/modificación/listado.